

Deflation-PINNs: Learning Multiple Solutions for PDEs and Landau De Gennes

Sean Disarò*, Ruma Rani Maity[†], Aras Bacho[‡]

Abstract

Nonlinear Partial Differential Equations (PDEs) are ubiquitous in mathematical physics and engineering. Although Physics-Informed Neural Networks (PINNs) have emerged as a powerful tool for solving PDE problems, they typically struggle to identify multiple distinct solutions, since they are designed to find one solution at a time. To address this limitation, we introduce Deflation-PINNs, a novel framework that integrates a deflation loss with an architecture based on PINNs and Deep Operator Networks (DeepONets).

By incorporating a deflation term into the loss function, our method systematically forces the Deflation-PINN to seek and converge upon distinct solution branches. We provide theoretical evidence on the convergence of our model and demonstrate the efficacy of Deflation-PINNs through numerical experiments on the Landau-de Gennes model of liquid crystals, a system renowned for its complex energy landscape and multiple equilibrium states. Our results show that Deflation-PINNs can successfully identify and characterize multiple distinct crystal structures.

Mathematics Subject Classification Physics-Informed Machine Learning, Neural Networks, DeepONets, Nonlinear Partial Differential Equations, Landau De Gennes

*seandisaro@gmail.com

[†]rumamaity081@gmail.com

[‡]bacho@caltech.edu. A.Bacho acknowledges support from the Air Force Office of Scientific Research (AFOSR) under awards FA9550-20-1-0358 (MURI: “Machine Learning and Physics-Based Modeling and Simulation”) and FA9550-24-1-0237, the U.S. Department of Energy (DOE), Office of Science, Office of Advanced Scientific Computing Research (ASCR) under award DE-SC0023163 (SEA-CROGS: “Scalable, Efficient and Accelerated Causal Reasoning Operators, Graphs and Spikes for Earth and Embedded Systems”), and the Office of Naval Research (ONR) under award N00014-25-1-2035.

1 Introduction

Partial Differential Equations (PDEs) have a wide range of applications in many disciplines. Especially in physics, all major theories are formulated in the language of PDEs. However, not all PDEs can be solved analytically. Thus, it is important to improve and provide new numerical methods to find approximated solutions for PDEs, making it possible to apply these solutions in engineering and other scientific fields.

With increasing computational power and with the rise of machine learning, there have been great ideas for solving PDEs with neural networks. Such methods include the Deep Ritz method [EY17], the Deep Galerkin method [SiS18], Physics-Informed Neural Networks (PINNs) [RPK19, RPK17a, RPK17b] and Variational PINNs (VPINNs) [KZK19].

A notoriously hard-to-compute field in which it is necessary to solve PDEs numerically is the Landau-De Gennes (LDG) theory ([MaZ09]), which is used to describe liquid crystals. In this context, some formulations of problems have a discrete number of solutions, which represent liquid crystal structures.

In this paper, we will explore a new method, which is based on Physics-Informed Machine Learning (PIML) and which is designed to find solutions to a LDG problem, which was also investigated in [LME12]. Our framework is designed to be fully unsupervised and is designed to find all the solutions of the LDG problem at once. It works by combining a PIML loss with a novel loss function, which ensures that the solutions found are distinct from each other. Furthermore, we provide theoretical evidence, which proofs that the architecture for our model is universal, by designing it in such a way that it is a special case of DeepONets (a neural network framework for learning operators between function spaces), which are known to be universal; [LJ*21, ChC95].

1.1 Related Work

There has already been research which aims to find a finite number of solutions for a PDE. We give a brief summary of related work.

HomPINNs

In [ZH*24] we present the so-called Homotopy Physics-Informed Neural Networks (HomPINNs), which are similar to what we want. They are designed to allow for a discrete number of solutions for a PDE. Their approach, however, aims to fit data from observations and additionally incorporating a Physics-Informed loss in their total loss. Thus, this can be viewed as a semi-supervised setting.

Learning Multiple Solutions from Random Initializations

In [ZWK25] the authors explore a different setting, in which they train many conventional Physics-Informed Neural Networks (PINNs) and they initialize them with different random parameters in the hope that through this random initialization, the optimization process yields different solutions. Afterwards, the different solutions that were obtained were analyzed and compared to see which solution was found how often. The advantage of this approach is that it is easily scalable as one can implement many PINNs in parallel. However, since we cannot guarantee that the different solutions the PINNs find are distinct, this approach is also very prone to redundancy and may lead to non-optimal use of resources.

Neural Networks for Nematic Liquid Crystals

In [SMZ24], the authors solve a Landau-De Gennes (LDG) problem with neural networks using a loss function based on an energy functional. However, unlike our approach, this approach is not qualified to distinguish and find multiple solutions for an LDG problem. In [SL*20], the authors use Convolutional Neural Networks (CNNs), to learn properties about liquid crystals. As the use of CNNs suggests, this work uses pictures of liquid crystals as inputs to predict properties like the order parameter of simulated nematic liquid crystals.

DeepONets

Deep Operator Networks (DeepONets) are a neural network framework for learning operators between function spaces. They were first introduced by [LJ*21] and have since been thoroughly studied in research. For a brief introduction to DeepONets, we refer to the appendix or to [LMK22] for a detailed survey on DeepONets.

1.2 Overview

In Section 2 we briefly recall the concepts of physics-informed machine learning. Then we proceed with presenting model architectures that are implemented with a hard constraint for Dirichlet conditions. In this context, we briefly discuss our method of doing so, by using radial extrapolation of functions. In the following subsection 2.3, we simplify the DeepONet architecture to fit our needs, and we argue that, for our purposes, we still maintain universality for the simplified model. More precisely, the simplified model will be defined in a way that it can only approximate a fixed number of functions. Moreover, we will introduce a new loss term, which makes the different solutions, which we want to approximate with our model, distinguishable. We call this new loss term **Deflation loss** and the total model will be called a **Deflation-PINN**. We will also provide a theoretical analysis on the approximation capabilities of our new model.

In the next section, we quickly recall the Landau-de Gennes model and introduce the concrete problem which we want to solve with our Deflation-PINN. We compare the solutions found with the Finite Element Method (FEM) and report the results.

Finally, in the last section we give a brief discussion of the paper.

2 Methodology

In this section we discuss the model which we use in this paper, and for this we will recall fundamental concepts including Physics-Informed Machine Learning, neural networks with hard constraints, and DeepONets.

2.1 Physics-Informed Machine Learning (PIML)

The Idea of *Physics-Informed Machine Learning (PIML)* was first presented in [RPK19]. The goal is to encode a PDE problem into a loss function, i.e let

$$\begin{aligned} \mathcal{D}[u](x) &= 0, & \text{for } x \in \Omega \\ \mathcal{B}[u](y) &= \psi(y), & \text{for } y \in \partial\Omega, \end{aligned} \quad (2.1)$$

where $\Omega \subset \mathbb{R}^d$ is compact and $\mathcal{D}, \mathcal{B}$ are differential operators and boundary operators, $u : \Omega \rightarrow \mathbb{R}^m, m \in \mathbb{N}$ is the solution of the PDE, $\psi : \partial\Omega \times [0, T] \rightarrow \mathbb{R}^m$ specifies the boundary condition. Then we can define the *generalization error* for 2.1 as

$$\mathcal{E}_G(\theta)^2 := \alpha_1 \cdot \int_{\Omega} |\mathcal{D}[f](x)|^2 dx + \alpha_2 \cdot \int_{\partial\Omega} |\mathcal{B}[f](x) - \psi(x)|^2 dx \quad (2.2)$$

α_1, α_2 are hyperparameters. The PIML framework seeks to minimize the difference between a model u_θ and the actual solution u through the loss function above. In other words, if the solution to 2.1 is unique, then the generalization error is zero if and only if $u_\theta = u$. This is due to the fundamental lemma of the calculus of variation.

When the model used is a neural network, then the respective neural network with PIML loss is usually referred to as a *Physics-Informed Neural Network (PINN)*.

To actually compute the generalization error $\mathcal{E}_G(\theta)$, we need to discretize them. The typical framework for this is to first discretize the differential operators in $\mathcal{R}_i[\mathbf{f}](\mathbf{x})$ with automatic differentiation (AD) ([GrW08]) and then discretize the three integrals. For the discretization of the integrals, one can use e.g. a Riemann sum. For more literature on PIML, we refer to [KK*21, WS*23?].

2.2 Hard Constraints For Boundary Conditions

Usually, when a loss function consists of multiple objectives, e.g., such as the three integrals in $\mathcal{E}_G$, it is not trivial how to choose the respective hyperparameter α_i . This was discussed in more detail in [EsR23, TK*24]. In such multi-objective loss functions, it is desirable to simplify the problem so that we have fewer objectives to approximate.

One way to achieve this in the PINN framework is to adapt the architecture of the neural network such that it automatically satisfies the boundary conditions, and thus the generalization error reduces to only $\mathcal{E}_G(\theta)^2 := \int_{\Omega} |\mathcal{R}_i[u_\theta](x)|^2 dx$. To ensure this, we can choose the following ansatz.

$$\tilde{f}_\theta(x) := \omega(x) \cdot NN_\theta(x) + b(x) \approx u(x)$$

where $\omega > 0$ on the interior $\text{int}(\Omega)$ and $\omega = 0$ on $\partial\Omega$. $b : \bar{\Omega} = \partial\Omega \cup \Omega \rightarrow \mathbb{R}$ should be at least continuous and chosen so that $b|_{\partial\Omega} = \psi$. $NN_\theta(x)$ is a neural network and the approximation is true for a suitable choice of parameters θ . Note that $\tilde{f}_\theta$ satisfies the exact Dirichlet condition independently of the choice of θ .

The exact imposition of boundary constraints has been discussed abundantly in literature, see [SuS22a, LeM20, LP*21?].

In this paper we use a custom extension which works in general on open, bounded star domains. The reason is that we found that this extension worked well for our experiments. The extension is constructed radially with respect to the center of the star domain.

So, let us assume that we have a bounded, open, and star shaped domain $\Omega \subset \mathbb{R}^d$. We can consider Ω in spherical coordinates with center x_0 , i.e., for a point $x \in \Omega$, which is given in cartesian coordinates, we view this as

$$\begin{aligned} x &\cong (\varphi_x^{x_0}, \theta_x^{x_0}, r_x^{x_0}) := (\varphi_{x-x_0}^0, \theta_{x-x_0}^0, r_{x-x_0}^0) \\ &:= (\varphi_{x-x_0}, \theta_{x-x_0}, \|x - x_0\|_2) \in [0, 2\pi) \times [0, \pi)^{d-2} \times [0, \infty), \end{aligned}$$

where $(\varphi_{x-x_0}, \theta_{x-x_0}, \|x - x_0\|_2) \in [0, 2\pi) \times [0, \pi)^{d-2} \times [0, \infty)$ is the usual representation of the point $x - x_0$ in spherical coordinates. Note that each boundary point $x \in \partial\Omega$ corresponds to one and only one angle in spherical coordinates, i.e. $\partial\Omega \longleftrightarrow [0, 2\pi) \times [0, \pi)^{d-2} \longleftrightarrow \mathbb{S}^1 \times \mathbb{S}^{d-2}$, where $\mathbb{S}^n = \partial B(0, 1) \subset \mathbb{R}^{n+1}$ denotes the boundary of the unit ball in $\mathbb{R}^{n+1}$. Furthermore, it is easy to see that these mappings are all continuous. This gives rise to the following definition.

Definition 1 Let $\Omega \subset \mathbb{R}^d$ be a bounded, open and star shaped domain with respect to $x_0 \in \Omega$. We define the radial boundary distance function (RBD function) $r_{\Omega, x_0} : \partial\Omega \rightarrow (0, \infty)$ of Ω with respect to x_0 , as the function that satisfies for all $x_b \in \partial\Omega \cong [0, 2\pi) \times [0, \pi)^{d-2}$ and $x \in \Omega \setminus \{x_0\}$, such that $(\varphi_x^{x_0}, \theta_x^{x_0}) = (\varphi_{x_b}^{x_0}, \theta_{x_b}^{x_0})$, that

$$r_{\Omega, x_0}(x) := r_{\Omega, x_0}(\varphi_x^{x_0}, \theta_x^{x_0}, r_x^{x_0}) := r_{\Omega, x_0}(\varphi_{x_b}^{x_0}, \theta_{x_b}^{x_0}, r_{x_b}^{x_0}) := r_{\Omega, x_0}(\varphi_{x_b}^{x_0}, \theta_{x_b}^{x_0}) := \|x_b - x_0\|_2.$$

The definition takes advantage of a slight abuse of notation, as the RBD function is formulated in terms of cartesian and x_0 centered spherical coordinates. The definition is to be understood so that every point (except the center) is mapped radially to a boundary point, and the function returns the distance from the center to that boundary point.

Example 2.1 The ball $B(x_0, R) \subset \mathbb{R}^d$ with radius $R > 0$ and center x_0 is a strongly star-shaped domain with respect to x_0 . Its RBD function is given by $r_{B(x_0, R), x_0} : x \mapsto R$ for all $x \in \partial B(x_0, R)$.

Now, it is easy to define spherical continuations in terms of the RBD function of a domain.

Definition 2 Let $\Omega \subset \mathbb{R}^d$ be a bounded, open and star shaped domain with respect to x_0 and let r_{Ω, x_0} be a RBD function for Ω . Let $h : [0, 1] \rightarrow [0, 1]$ be a continuous function such that $h|_{[0, 1]} > 0$, $h(0) = 1$ and $h(1) = 0$. Let $f \in C(\partial\Omega)$. Define

$$\tilde{\omega}(x) : \begin{cases} \Omega \rightarrow [0, 1] \\ x \mapsto \begin{cases} h(\frac{\|x - x_0\|}{r_{\Omega, x_0}(x)}), & \text{if } x \neq x_0 \\ 0, & \text{if } x = x_0. \end{cases} \end{cases} \quad \text{and} \quad \tilde{f} : \begin{cases} \Omega \rightarrow \mathbb{R} \\ x \mapsto f(\varphi_x^{x_0}, \theta_x^{x_0}) \cdot (1 - \tilde{\omega}(x)). \end{cases}$$

Then $\tilde{\omega}, \tilde{f} \in C(\bar{\Omega})$, we have $\tilde{\omega} = 0$ on $\partial\Omega$ and $\tilde{\omega} > 0$ on Ω and $\tilde{f}|_{\partial\Omega} = f$.

If we choose $h : [0, 1] \rightarrow [0, 1]$ to be smooth and if Ω and $f : \partial\Omega \rightarrow \mathbb{R}$ are smooth, then $\tilde{\omega}$ and $\tilde{f}$ are smooth. The general difficulty of this approach is to find an RBD function for the domain, which is in general not easy.

Note that $\tilde{\omega}$ satisfies the condition we need above for our ansatz. However, we will not use this $\tilde{\omega}$, but rather only $\tilde{f}$ for the extrapolation of the boundary function. For ω we will use the C^∞ -construction from [LM*21].

2.3 Deflation-PINNs

There are PDE problems, which have a finite number of solutions $K > 1$. For an example of such a problem, we refer to the section where we present our numerical results. However, this kind of problem is not covered by the classical PINN framework, as it is only suitable for learning one solution at a time. Although the DeepONet framework can learn multiple solutions for a class of PDE problems, it is unlikely to be the most efficient choice, since it is built to learn infinitely many solutions in the form of an operator between function spaces. Thus, it may be seen as an overkill if we only want finitely many solutions. This section introduces *Deflation-PINNs*, i.e. our new method for learning multiple solutions with neural networks. The method consists of two new keypoints.

1. We will introduce a neural network architecture to learn multiple solutions at the same time. The architecture will be a special case of the DeepONet architecture but designed in such a way that it only admits a finite number of solutions.
2. We will introduce a loss function that ensures that we learn different solutions for the same problem.

1) Architecture

We start with (unstacked) DeepONets, which want to approximate an operator over a function space, i.e. they map $G : \mathcal{A} \times \mathbb{R}^d \rightarrow \mathbb{R}$, where $\mathcal{A}$ is a space of functions. The output then is of the form $\sum_{i=1}^p \tau_i(x) \cdot \beta_i(f) \approx G(f)(x)$ where $x \in \mathbb{R}^d$ and $f \in \mathcal{A}$. The τ_i are the outputs of a neural network $x \mapsto (\tau_1(x), \dots, \tau_p(x))$. The β_i use the evaluation of the function at previously fixed sensor points, that is, take $(\tilde{x}_i)_{i=1}^N \in \text{dom}(f)$ sensor points, then

$$f \mapsto (\beta_1(f), \dots, \beta_p(f)) = (\beta_1(f(\tilde{x}_1, \dots, \tilde{x}_N)), \dots, \beta_p(f(\tilde{x}_1, \dots, \tilde{x}_N)))$$

where $\beta : \mathbb{R}^N \rightarrow \mathbb{R}^p$ is a neural network.

However, in our setting, we want to learn only $K \in \mathbb{N}$ solutions that do not depend on an input function. Thus, we propose, instead of using a neural network $\beta_i : \mathbb{R}^N \rightarrow \mathbb{R}^p$, to use only K feature vectors of size p (one feature vector for each solution), which encode the respective function, i.e., we want $\beta^k = (\beta_1^k, \dots, \beta_p^k) \in \mathbb{R}^p$. These feature vectors can then be trained as part of the model's parameters. In total, our solution is of the form

$$\tilde{G}(k, x) := \sum_{i=1}^p \tau_i(x) \cdot \beta_i^k \approx u_k(x),$$

where $x \in \text{dom}(u_k)$ and u_k for $k = 1, \dots, K$ are the K solutions we want to approximate and τ_i are still neural networks.

The advantage of using fixed feature $\beta_k \in \mathbb{R}^p$ instead of neural networks $\beta : \mathcal{A} \rightarrow \mathbb{R}^p$ is that we only need $K \cdot p$ parameters to represent the K functions. We call the $\beta_k \in \mathbb{R}^p$ **Branch Weights**.

If we were to use the DeepONet architecture, then we would need at least $S \cdot p + S$ parameters, where $S \in \mathbb{N}$ is the number of sensor points, which can easily be greater than the number of solutions, that is, $S > K$. Furthermore, the K feature vector directly incorporates the solution structure such that it can be decoded by the architecture, since the feature vectors are trained to do so, which is in contrast to the fixed sensor points, which cannot be changed.

Note that since we have a finite number of solutions, which we want to approximate for our architecture, we still want to approximate a map of compact sets, and thus we can derive universality for this architecture as a special case of theorem 2.

Corollary 1 *Suppose that $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ is a continuous and non-polynomial activation function, X is a Banach function space, $u_1, \dots, u_N \in X$ and $K \subset \mathbb{R}^d$ compact.*

Then for all $\varepsilon > 0$ there exists $p \in \mathbb{N}$, $\zeta_i \in \mathbb{R}$, $\omega_i \in \mathbb{R}^d$, such that for all $n \in \{1, \dots, N\}$ and $y \in K$ we have that

$$\left| u_n(y) - \sum_{i=1}^p \beta_n^i \sigma(\omega_i \cdot y + \zeta_i) \right| < \varepsilon.$$

Proof. This is just a special case of theorem 2 □

2) Deflation Loss

We now have an architecture for our problem, but the question remains how to train this. With the PIML loss, we can learn true solutions. However, we still need a loss, which tells the model that the $K \in \mathbb{N}$ solutions it learns (with the PIML loss) need to be distant from each other/not the same. More precisely, the solutions u_k that we want to approximate have to be at least $d_{min} := \min_{i,j \in \{1, \dots, K\}, i \neq j} \|u_i - u_j\|_{L^2}$ apart.

Let $\tilde{G}$ with

$$\tilde{G}(k, x) := \sum_{i=1}^p \tau_i(x) \cdot \beta_i^k \approx u_k(x)$$

be our model. We can rewrite the condition that the solutions be at least d_{min} apart as the following loss function:

$$\mathcal{L}_{Def}(\tilde{G}) := \frac{2}{K \cdot (K-1)} \sum_{i=1}^K \sum_{j=i+1}^K \max(1 - \frac{1}{d_{min}} \|\tilde{G}(i, \cdot) - \tilde{G}(j, \cdot)\|_{L^2}, 0),$$

i.e., the respective approximated solutions $\tilde{G}(i, \cdot)$ have at least distance d_{min} between each other in the L^2 -norm if and only if $\mathcal{L}_{Deflation}(\tilde{G}) = 0$.

Thus, the total loss for training our model $\tilde{G}$ can be given as

$$\mathcal{L}_{total}(\tilde{G}) := \alpha \cdot \sum_{i=1}^K \mathcal{E}_G(\tilde{G}(i, \cdot)) + \beta \cdot \mathcal{L}_{Def}(\tilde{G})$$

where $\mathcal{E}_G$ is the PIML generalization error and for some hyperparameters $\alpha, \beta > 0$. We call this model $\tilde{G}$ together with the loss function $\mathcal{L}_{total}$ a **Deflation-PINN**. Figure 1 illustrates the architecture.

3) Hard Constraints Approximation Theory

Approximation theory for models with hard constraints is a topic which has already been discussed in literature; see e.g. [SuS22b], where the authors use the framework of R-functions, to establish a strong theoretical framework. Since we will also use hard constraints in the model for our experiments later, we want to invest a small section of this paper as well into justifying the use of such hard constraint models as described in section 2.2, via a novel elementary theorem. The theorem is formulated in the $\|\cdot\|_\infty$ -topology on the continuous functions, which makes it applicable in combination with corollary 1.

Theorem 1 Assume $\Omega \subset \mathbb{R}^d$ is an open domain. Let $g, h \in C(\bar{\Omega})$, s.t. $h|_{\partial\Omega} = 0$ and $g > 0$ on Ω , then there exists a sequence of functions $(f_n)_{n \in \mathbb{N}} \subset C(\bar{\Omega})$, s.t.

$$\|f_n g - h\|_{\Omega} = \|f_n g - h\|_{\infty, \Omega} \xrightarrow{n \rightarrow \infty} 0$$

We now apply the theorem above onto our Deflation-PINN architecture:

Corollary 2 Suppose that $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ is a continuous and non-polynomial activation function, $\Omega \subset \mathbb{R}^d$ is compact, $u_1, \dots, u_N \in C(\bar{\Omega})$, $g, h \in C(\bar{\Omega})$ s.t. $h|_{\partial\Omega} = u_n$ for $n \in \{1, \dots, N\}$ and $g|_{\partial\Omega} = 0$.

Then for all $\varepsilon > 0$ there exists $p \in \mathbb{N}$, $\zeta_i \in \mathbb{R}$, $\omega_i \in \mathbb{R}^d$, such that for all $n \in \{1, \dots, N\}$ and $y \in K$ we have that

$$\left| u_n(y) - \left(h(y) - g(y) \cdot \sum_{i=1}^p \beta_k^i \sigma(\omega_i \cdot y + \zeta_i) \right) \right| < \varepsilon.$$

Proof. Fix $\varepsilon > 0$. From theorem 1 we get for each $n \in \{1, \dots, N\}$ a $f_{\varepsilon, n} \in C^\infty$ such that

$$\|f_{\varepsilon, n} \cdot g - h\|_{\Omega} < \varepsilon.$$

Then all need to do is to apply corollary 1 and approximate each $f_{\varepsilon, n}$ such that

$$\left| f_{\varepsilon, n}(y) - \sum_{i=1}^p \beta_n^i \sigma(\omega_i \cdot y + \zeta_i) \right| < \varepsilon$$

for all $y \in K$. The rest follows from the triangle inequality. □

3 Numerical Experiments

In this section, we discuss numerical results on the example of the Reduced Two-Dimensional Landau-De Gennes model (LDG model) from uncoupled ferronematics/liquid crystal theory. Then we present the solutions we got from our Deflation-PINN. The code can be found in <https://github.com/SeanDisaro/DeflationPINNs>.

3.1 Landau-De Gennes Model

Landau-de Gennes theory is a phenomenological variational theory that describes the state of nematic liquid crystals in terms of an order parameter, $\mathbf{Q}$ tensor. In two dimensions, $\mathbf{Q}$ tensor is a symmetric traceless matrix in $\mathbb{R}^{2 \times 2}$, and is represented as a vector $\mathbf{Q} := (Q_{11}, Q_{12})$. In the absence of surface effects and external field, the reduced two dimensional LdG [LME12] functional in the dimensionless form is given by

$$\mathcal{E}(\mathbf{Q}) = \int_{\Omega} (|\nabla \mathbf{Q}|^2 + \epsilon^{-2}(|\mathbf{Q}|^2 - 1)^2) dx, \quad (3.1)$$

where $\mathbf{Q} \in \mathbf{H}^1(\Omega)$ and ϵ is a small parameter that depends on the elastic constant, the bulk energy parameters, and the domain size. We are concerned with the minimization of the functional $\mathcal{E}(\bullet)$ for the Lipschitz continuous boundary function $\mathbf{Q}_b : \partial\Omega \rightarrow \mathbb{R}^2$ consistent with the experimentally imposed tangent boundary conditions. The energy formulation in (3.1) seeks $\mathbf{Q} \in \mathbf{H}^1(\Omega)$ such that

$$-\Delta \mathbf{Q} = 2\epsilon^{-2}(1 - |\mathbf{Q}|^2)\mathbf{Q} \text{ in } \Omega \text{ and } \mathbf{Q} = \mathbf{Q}_b \text{ on } \partial\Omega. \quad (3.2)$$

For our experiment, we use $\epsilon = 0.02$. We solve the system (3.2) in $\Omega := [0, 1] \times [0, 1]$ with the Dirichlet boundary condition $\mathbf{Q}_b$ constructed utilizing trapezoidal shape function $T_d : [0, 1] \rightarrow \mathbb{R}$ with $d = 3\epsilon$ as

$$\mathbf{Q}_b = \begin{cases} (T_d(x), 0) & \text{on } y = 0 \text{ and } y = 1, \\ (-T_d(y), 0) & \text{on } x = 0 \text{ and } x = 1, \end{cases} \text{ and } T_d(t) = \begin{cases} t/d, & 0 \leq t \leq d, \\ 1, & d \leq t \leq 1-d, \\ (1-t)/d, & 1-d \leq t \leq 1. \end{cases}$$

Experimental and numerical investigations suggest that there are six stable solutions to this problem on a square domain, which have been computed using FEMs; see [TD*07, LME12, MMN21a]. Two classes of stable experimentally observable configurations are reported: diagonal states (D1 and D2) in which the nematic directors are aligned along the square diagonals, and rotated states (R1, R2, R3, R4) in which the nematic directors rotate in π radian angles across the square edges. Some of these nematic equilibria are presented in Figure ??.

3.2 Details About The Experiment

We want to find the six solutions from the previous section. The experiments were performed on an RTX 3060-Ti GPU with 8GB VRAM. We use the model discussed in Section 2.3 with a hard constraint for the Dirichlet condition. We need to adjust it such that it has two outputs, i.e. the final model $\mathbf{G} = (G^1, G^2)$ is of the form

$$\tilde{G}^1(k, x) \cdot \omega(x) + \tilde{Q}_b(x) = \omega(x) \cdot \sum_{i=1}^p \tau_i(x) \cdot \beta_i^k + \tilde{Q}_b(x) \approx u_k^1(x)$$

$$\tilde{G}^2(k, x) \cdot \omega(x) = \omega(x) \cdot \sum_{i=p+1}^{2p} \tau_i(x) \cdot \beta_i^k \approx u_k^2(x),$$

where $\beta^k \in \mathbb{R}^p$ for $k = 1, \dots, 6$ and $\tau : \mathbb{R}^2 \rightarrow \mathbb{R}^p$ is a neural network with skip connections. $\tilde{G} = (\tilde{G}^1, \tilde{G}^2)$ is the Def-PINN model from Section 2.3 without the hard constraint for two outputs. $\omega : [0, 1]^2 \rightarrow \mathbb{R}$ is such that $\omega > 0$ on $(0, 1)^2$ and $\omega = 0$ on $\partial[0, 1]^2$. $\tilde{Q}_b(x)$ is an extension of the trapezoidal function used for the boundary condition from the last section, i.e.

$$\tilde{Q}_b(x)|_{\partial[0,1]^2} = \mathbf{Q}_b^1 = \begin{cases} T_d(x) & \text{on } y = 0 \text{ and } y = 1, \\ -T_d(y) & \text{on } x = 0 \text{ and } x = 1. \end{cases}$$

Note that we did not add anything to the second output term of the model and multiplied it only by ω . This is because we require the Dirichlet condition to be zero for the second term. Concretely, we use the C^∞ -construction from [LM*21] for ω and for $\tilde{Q}_b(x)$ we use the radial extension from Definition 2, as described in Section 2.2. For this, we use the point $(\frac{1}{2}, \frac{1}{2}) \in \Omega := [0, 1]^2$ as a center for our star domain. The exact formula for the RBD function can be explicitly calculated and for this we refer to our implementation in our repository <https://github.com/SeanDisaro/DeflationPINNs>. For the function $h : [0, 1] \rightarrow [0, 1]$ from Definition 2 we simply use the function $h(x) = 1 - x$, since this is very efficient to compute.

Regarding the PIML loss function, we only need to implement the dynamics of the model, since we already satisfy the Dirichlet condition by construction. Thus we can use equation 3.2 to formulate a generalization error of the form

$$\begin{aligned} \mathcal{E}_G(\mathbf{G}) := & \int_{\Omega} \left| \epsilon^2 \cdot \Delta G^1 + 2 \cdot (1 - G^1 \cdot G^1 - G^2 \cdot G^2) \cdot G^1 \right| dx \\ & + \int_{\Omega} \left| \epsilon^2 \cdot \Delta G^2 + 2 \cdot (1 - G^1 \cdot G^1 - G^2 \cdot G^2) \cdot G^2 \right| dx. \end{aligned}$$

We evaluated the PIML loss on collocation points on a 33×33 -grid and took the Riemann sum for this grid. The grid covers the domain $[0 + \delta, 1 - \delta]^2$. The safety distance is to facilitate training and we can safely do this, since the boundary condition is enforced.

Regarding the deflation loss, we use one additional trick. Instead of computing the $L^2([0, 1]^2, \mathbb{R}^2)$ norm for the model $\mathbf{G}(i, \cdot) : \mathbb{R}^2 \rightarrow \mathbb{R}^2$, which is suggested by how we introduced the deflation loss in Section 2.3, we only compute the $L^2([0, 1]^2, \mathbb{R})$ norm for our second component. There is no particular reason why we chose the second component, but the advantage of only using one component is that we make the computation of the deflation loss more efficient. Additionally, this simpler form has shown to facilitate the training process in our experiments.

We used $d_{min} = 0.4$. This parameter was found empirically through testing and was not computed analytically. This was done by checking if the solutions after training were too similar (in which case we increased d_{min}) and, on the other hand, if the PIML term did not converge, then we decreased d_{min} . Thus, the deflation loss we used is given by

$$\mathcal{L}_{Def}(\mathbf{G}) := \frac{1}{15} \sum_{i=1}^6 \sum_{j=i+1}^6 \max(1 - \frac{5}{2} \|G^2(i, \cdot) - G^2(j, \cdot)\|_{L^2([0,1]^2, \mathbb{R})}, 0).$$

To compute the $L^2([0, 1]^2, \mathbb{R})$ norm for the deflation loss, we use the same grid / collocation points, as for the PIML loss. The total loss is of the form

$$\mathcal{L}_{total}(\mathbf{G}) := \alpha \cdot \sum_{i=1}^6 \mathcal{E}_G(\mathbf{G}(i, \cdot)) + \beta \cdot \mathcal{L}_{Def}(\mathbf{G})$$

and we used $\alpha = 0.02$ and $\beta = 2$.

In our experiments, we use the Adam optimizer to train for 10,000 epochs with an initial learning rate of $1 \cdot 10^{-3}$. $\tau : \mathbb{R}^2 \rightarrow \mathbb{R}^p$ is of width 4000 and has one layer. We use tanh as the activation function. We set the output size of τ and the size of the representation vectors to $p = 8$.

We compare our results with the results from [MMN21b], which were calculated using the finite element method. As a metric of comparison, we use the relative L^2 -distances between the respective solutions from [MMN21b] and the solutions from our Deflation-PINN. The relative L^2 -distances can be found in table ?? . The relative L^2 -distances to the solutions from [MMN21b] is given by

$$\frac{\left\| \begin{pmatrix} Q_{11}^{NN} - Q_{11}^{FEM} \\ Q_{12}^{NN} - Q_{12}^{FEM} \end{pmatrix} \right\|_{L^2([0,1]^2, \mathbb{R}^2)}}{\left\| \begin{pmatrix} Q_{11}^{FEM} \\ Q_{12}^{FEM} \end{pmatrix} \right\|_{L^2([0,1]^2, \mathbb{R}^2)}},$$

where $(Q_{11}^{NN}, Q_{12}^{NN})$ is the solution from the Deflation-PINN and $(Q_{11}^{FEM}, Q_{12}^{FEM})$ is the solution from [MMN21b] using the finite element method.

4 Discussion

In this paper, we discuss a Deflation-PINNs for solving nonlinear PDEs with a discrete number of solutions. We based our approach on well established models such as PINNs and DeepONets. More precisely, we simplified the DeepONet architecture so that it can only approximate a finite discrete number of solutions, and we augmented the PINN loss with a deflation loss so that if it is provided with multiple solutions, the model can distinguish between different solutions. Contrary to other neural network approaches for finding multiple PDE solutions, we do not rely on random initializations and we showed that the deflation loss provides a robust and systematic method for finding multiple solutions. This method was tested on a Landau De Gennes problem and it was able to detect all solutions, which could also be detected with the classical finite element method.

References

- [ChC95] T. CHEN and H. CHEN. Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems. *IEEE Transactions on Neural Networks*, 6(4), 911–917, 1995.
- [EsR23] P. ESCAPIL-INCHAUSPÉ and G. A. RUZ. Hyper-parameter tuning of physics-informed neural networks: Application to helmholtz problems. *Neurocomputing*, 561, 126826, 2023.
- [EY17] W. E and B. YU. The deep ritz method: A deep learning-based numerical algorithm for solving variational problems. *CoRR*, abs/1710.00211, 2017.
- [GrW08] A. GRIEWANK and A. WALTHER. *Evaluating Derivatives*. Society for Industrial and Applied Mathematics, second edition, 2008.
- [KK*21] G. E. KARNIADAKIS, I. G. KEVREKIDIS, L. LU, P. PERDIKARIS, S. WANG, and L. YANG. Physics-informed machine learning. *Nature Reviews Physics*, 3(6), 422–440, Jun 2021.
- [KZK19] E. KHARAZMI, Z. ZHANG, and G. E. KARNIADAKIS. Variational physics-informed neural networks for solving partial differential equations. *CoRR*, 2019.
- [LeM20] C. LEAKE and D. MORTARI. Deep theory of functional connections: A new method for estimating the solutions of partial differential equations. *Machine Learning and Knowledge Extraction*, 2(1), 37–55, March 2020.
- [LJ*21] L. LU, P. JIN, G. PANG, Z. ZHANG, and G. E. KARNIADAKIS. Learning nonlinear operators via deepoNet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3), 218–229, March 2021.
- [LM*21] L. LU, X. MENG, Z. MAO, and G. E. KARNIADAKIS. DeepXDE: A deep learning library for solving differential equations. *SIAM Review*, 63(1), 208–228, 2021.
- [LME12] C. LUO, A. MAJUMDAR, and R. ERBAN. Multistability in planar liquid crystal wells. *Physics Review E*, 85, 061702, Jun 2012.
- [LMK22] S. LANTHALER, S. MISHRA, and G. E. KARNIADAKIS. Error estimates for deepoNets: A deep learning framework in infinite dimensions, 2022.
- [LP*21] L. LU, R. PESTOURIE, W. YAO, Z. WANG, F. VERDUGO, and S. G. JOHNSON. Physics-informed neural networks with hard constraints for inverse design. *SIAM Journal on Scientific Computing*, 43(6), B1105–B1132, 2021.
- [MaZ09] A. MAJUMDAR and A. ZARNESCU. Landau–de Gennes theory of nematic liquid crystals: the oseen–frank limit and beyond. *Arch. Ration. Mech. Anal.*, 196(1), 227–280, July 2009.
- [MMN21a] R. R. MAITY, A. MAJUMDAR, and N. NATARAJ. Discontinuous Galerkin finite element methods for the Landau-de Gennes minimization problem of liquid crystals. *IMA J. Numer. Anal.*, 41(2), 1130–1163, 2021.
- [MMN21b] R. R. MAITY, A. MAJUMDAR, and N. NATARAJ. Parameter dependent finite element analysis for ferronematics solutions. *Comput. Math. Appl.*, 103, 127–155, 2021.
- [RPK17a] M. RAISSI, P. PERDIKARIS, and G. E. KARNIADAKIS. Physics informed deep learning (part i): Data-driven solutions of nonlinear partial differential equations, 2017.
- [RPK17b] M. RAISSI, P. PERDIKARIS, and G. E. KARNIADAKIS. Physics informed deep learning (part ii): Data-driven discovery of nonlinear partial differential equations, 2017.

- [RPK19] M. RAISSI, P. PERDIKARIS, and G. KARNIADAKIS. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *J. Comput. Phys.*, 378, 686–707, 2019.
- [SiS18] J. SIRIGNANO and K. SPILIOPOULOS. Dgm: A deep learning algorithm for solving partial differential equations. *Journal of Computational Physics*, 375, 1339–1364, December 2018.
- [SL*20] H. Y. D. SIGAKI, E. K. LENZI, R. S. ZOLA, M. PERC, and H. V. RIBEIRO. Learning physical properties of liquid crystals with deep convolutional neural networks. *Scientific Reports*, 10(1), 7664, May 2020.
- [SMZ24] B. SHI, A. MAJUMDAR, and L. ZHANG. Neural network-based tensor model for nematic liquid crystals with accurate microscopic information, 2024.
- [SuS22a] N. SUKUMAR and A. SRIVASTAVA. Exact imposition of boundary conditions with distance functions in physics-informed deep neural networks. *Comput. Methods Appl. Mech. Eng.*, 389, 114333, 2022.
- [SuS22b] N. SUKUMAR and A. SRIVASTAVA. Exact imposition of boundary conditions with distance functions in physics-informed deep neural networks. *Computer Methods in Applied Mechanics and Engineering*, 389, 114333, 2022.
- [TD*07] C. TSAKONAS, A. J. DAVIDSON, C. V. BROWN, and N. J. MOTTRAM. Multistable alignment states in nematic liquid crystal filled wells. *Applied Physics Letters*, 90, Article 111913, 2007.
- [TK*24] J. D. TOSCANO, T. KÄUFER, Z. WANG, M. MAXEY, C. CIERPKA, and G. E. KARNIADAKIS. Inferring turbulent velocity and temperature fields and their statistics from lagrangian velocity measurements using physics-informed kolmogorov-arnold networks, 2024.
- [WS*23] S. WANG, S. SANKARAN, H. WANG, and P. PERDIKARIS. An expert’s guide to training physics-informed neural networks, 2023.
- [ZH*24] H. ZHENG, Y. HUANG, Z. HUANG, W. HAO, and G. LIN. Hompinns: Homotopy physics-informed neural networks for solving the inverse problems of nonlinear differential equations with multiple solutions. *J. Comput. Phys.*, 500, 112751, 2024.
- [ZWK25] Z. ZOU, Z. WANG, and G. E. KARNIADAKIS. Learning and discovering multiple solutions using physics-informed neural networks with random initialization and deep ensemble, 2025.

A Theorems and Proofs

Theorem 2 ([ChC95]) *Suppose that*

- $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ *is continuous and non-polynomial activation function,*
- X *is a Banach space,*
- $K_1 \subset X$, $K_2 \subset \mathbb{R}^d$ *are both compact,*
- $V \subset C(K_1)$ *is compact,*
- $G : V \rightarrow C(K_2)$ *be a continuous (non-linear) operator,*

then for all $\varepsilon > 0$ there exists $n, p, m \in \mathbb{N}$, $c_i^k, \xi_{ij}^k, \theta_i^k, \zeta_k \in \mathbb{R}$, $\omega_k \in \mathbb{R}^d$, $x_j \in K_1$ such that for all $u \in V$ and $y \in K_2$ we have that

$$\left| G(u)(y) - \sum_{k=1}^p \sum_{i=1}^n c_i^k \sigma \left(\sum_{j=1}^m \xi_{ij}^k u(x_j) + \theta_i^k \right) \sigma(\omega_k \cdot y + \zeta_k) \right| < \varepsilon.$$

Lemma 1 *For $n \in \mathbb{N}$ let $N_n \subset \mathbb{R}^d$ be compact sets. Assume that $N_n \supset N_{n+1}$ for all $n \in \mathbb{N}$ and that $N := \bigcup_{n \in \mathbb{N}} N_n \subset \mathbb{R}^d$ is bounded. Assume that $h \in C(\bar{N})$ and $h|_{\bigcap_{n \in \mathbb{N}} N_n} = 0$. Then*

$$\|h\|_{\infty, N_n} \xrightarrow{n \rightarrow \infty} 0$$

Proof. Assume $\exists C > 0$ s.t. $\forall n \in \mathbb{N} : \|h\|_{\infty, N_n} > 0$. Then we find a sequence of points $x_n \in N_n$, s.t. $|h(x_n)| = \sup_{x \in N_n} |h(x)| > C$. $(x_n)_{n \in \mathbb{N}} \subset N$ is a bounded sequence. Bolzano Weierstrass gives us a converging subsequence $(x_{n_k})_{k \in \mathbb{N}}$ and $x_0 := \lim_{k \rightarrow \infty} x_{n_k} \in \bigcap_{n \in \mathbb{N}} N_n$. Thus we found a $x_0 \in \bigcap_{n \in \mathbb{N}} N_n$ s.t. $h(x_0) \geq C > 0$. $\square$

Theorem 3 *Assume $\Omega \subset \mathbb{R}^d$ is an open domain. Let $g, h \in C(\bar{\Omega})$, s.t. $h|_{\partial\Omega} = 0$ and $g > 0$ on Ω , then there exists a sequence of functions $(f_n)_{n \in \mathbb{N}} \subset C(\bar{\Omega})$, s.t.*

$$\|f_n g - h\|_{\Omega} = \|f_n g - h\|_{\infty, \Omega} \xrightarrow{n \rightarrow \infty} 0$$

Proof. Fix $\varepsilon > 0$. We begin by defining for $\lambda \geq 0$

$$A_\lambda := \{x \in \Omega : \text{dist}(x, \partial\Omega) > \lambda\}$$

A_λ has the following properties:

- A_λ is open for all $\lambda \geq 0$
- $\exists \lambda > 0$ s.t. $A_\lambda \neq \emptyset$
- If $A_\lambda \neq \emptyset$, then $A_{\frac{\lambda}{2}} \supsetneq A_\lambda$

Now we observe, that for a fix $\lambda > 0$ s.t. $A_\lambda \neq \emptyset$ if we define $A^n := A_{\frac{\lambda}{2^n}}$ and $N_n := \bar{\Omega} \setminus A^n$ we have that $(N_n)_{n \in \mathbb{N}}$ fulfills the condition from lemma 1 above and thus we can find a $n \in \mathbb{N}$ s.t. $\|h\|_{\infty, N_n} < \varepsilon$. Now we fix this n and define

$$\begin{cases} A := A^n \\ B := A^{n+1} \end{cases}$$

Define the function $f(x) = \frac{h(x)}{g(x)}$ for $x \in \Omega$. Note that f is continuous on Ω , but not necessarily continuous on $\bar{\Omega}$. Now we take a continuous function $\varphi \in C(\bar{\Omega})$, s.t. $\|\varphi\| = 1$, $\varphi = 1$ on A and $\varphi = 0$ on $\bar{\Omega} \setminus B$. Define the function

$$f_\varepsilon := f \cdot \varphi \in C_0(\bar{\Omega}) = \{g \in C(\bar{\Omega}) : g|_{\partial\Omega} = 0\} \subset C(\bar{\Omega}).$$

The following simple estimate concludes the proof:

$$\begin{aligned} \|f_\varepsilon g - h\|_\Omega &\leq \underbrace{\|f_\varepsilon g - h\|_A}_{=\|(f_\varepsilon - f)g\|_A=0} + \|f_\varepsilon g - h\|_{\Omega \setminus A} \\ &\leq \|f_\varepsilon g\|_{\Omega \setminus A} + \underbrace{\|h\|_{\Omega \setminus A}}_{\leq \varepsilon} \\ &\leq \|f \varphi g\|_{\Omega \setminus A} + \varepsilon \\ &= \|f \varphi g\|_{B \setminus A} + \varepsilon \\ &\leq \|f g\|_{B \setminus A} + \varepsilon \\ &= \|h\|_{B \setminus A} + \varepsilon \\ &\leq \|h\|_{\Omega \setminus A} + \varepsilon \leq 2\varepsilon \end{aligned}$$

□

B DeepONets

Deep Operator Networks (DeepONets) are a neural network framework for learning operators between function spaces. They are based on the classical approximation result for operators by [ChC95].

Theorem 4 ([ChC95]) *Suppose that*

- $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ *is continuous and non-polynomial activation function,*
- X *is a Banach space,*
- $K_1 \subset X$, $K_2 \subset \mathbb{R}^d$ *are both compact,*
- $V \subset C(K_1)$ *is compact,*
- $G : V \rightarrow C(K_2)$ *be a continuous (non-linear) operator,*

then for all $\varepsilon > 0$ there exists $n, p, m \in \mathbb{N}$, $c_i^k, \xi_{ij}^k, \theta_i^k, \zeta_k \in \mathbb{R}$, $\omega_k \in \mathbb{R}^d$, $x_j \in K_1$ such that for all $u \in V$ and $y \in K_2$ we have that

$$\left| G(u)(y) - \sum_{k=1}^p \sum_{i=1}^n c_i^k \sigma \left(\sum_{j=1}^m \xi_{ij}^k u(x_j) + \theta_i^k \right) \sigma(\omega_k \cdot y + \zeta_k) \right| < \varepsilon.$$

*The fixed points x_j are called **sensor points**.*

[LJ*21] reused this idea to introduce what we now know as DeepONets. Let us now give a precise definition of DeepONets. This definition is borrowed from [LMK22].

Definition 3 Let $D \subset \mathbb{R}^d$ be a compact domain and $m, p \in \mathbb{N}$. Define the following operators

- **Encoder:** Given a set of sensor points $(x_j)_{j=1}^m \subset D$, we define the linear mapping

$$\begin{cases} \mathcal{E} : C(D) \rightarrow \mathbb{R}^m, \\ \mathcal{E}(u) = (u(x_1), \dots, u(x_m)) \end{cases}$$

- **Approximator:** Given the evaluation of the sensor points, the approximator is a neural network, that maps $\mathcal{A} : \mathbb{R}^m \rightarrow \mathbb{R}^p$. The composition $\mathcal{A} \circ \mathcal{E}$ is called **Branch Net**.

- **Reconstructor:** We introduce a neural network $\begin{cases} \tau : \mathbb{R}^n \rightarrow \mathbb{R}^{p+1} \\ y \mapsto (\tau_0(y), \dots, \tau_p(y)) \end{cases}$ and call it the **Trunk Net**. Then the reconstructor is defined as $\begin{cases} \mathcal{R} = \mathcal{R}_\tau : \mathbb{R}^p \rightarrow C(U), \\ \alpha \mapsto \tau_0(\cdot) + \sum_{k=1}^p \alpha_k \tau_k(\cdot) \end{cases}$.

Then call the composition of reconstructor and branch net $\mathcal{E} \circ \mathcal{A} \circ \mathcal{R}$ a **DeepONet**.

In the original paper [LJ*21], the authors propose two different implementations of the approximator. The first is the *unstacked DeepONet*, where the approximator consists of only one neural network, as in the above definition, and the second is the *stacked DeepONet*, where the approximator consists of one neural network for each output feature. More precisely, we get

$$\mathcal{A}_{\text{stacked}}(u_1, \dots, u_m) = (\mathcal{A}_1(u_1, \dots, u_m), \dots, \mathcal{A}_p(u_1, \dots, u_m))$$

where each $\mathcal{A}_i : \mathbb{R}^m \rightarrow \mathbb{R}$ is a neural network. Although this is the version used in theorem 2, the authors outline the drawbacks in computational complexity of the stacked version and even show in their experimental findings that the unstacked version outperforms the stacked version in terms of total error. Furthermore, it can be easily shown that we also get universality for the unstacked version, by using the fact that neural networks are universal approximators. Furthermore, for an extensive error analysis for DeepONets we refer to [LMK22].

C Figures And Tables

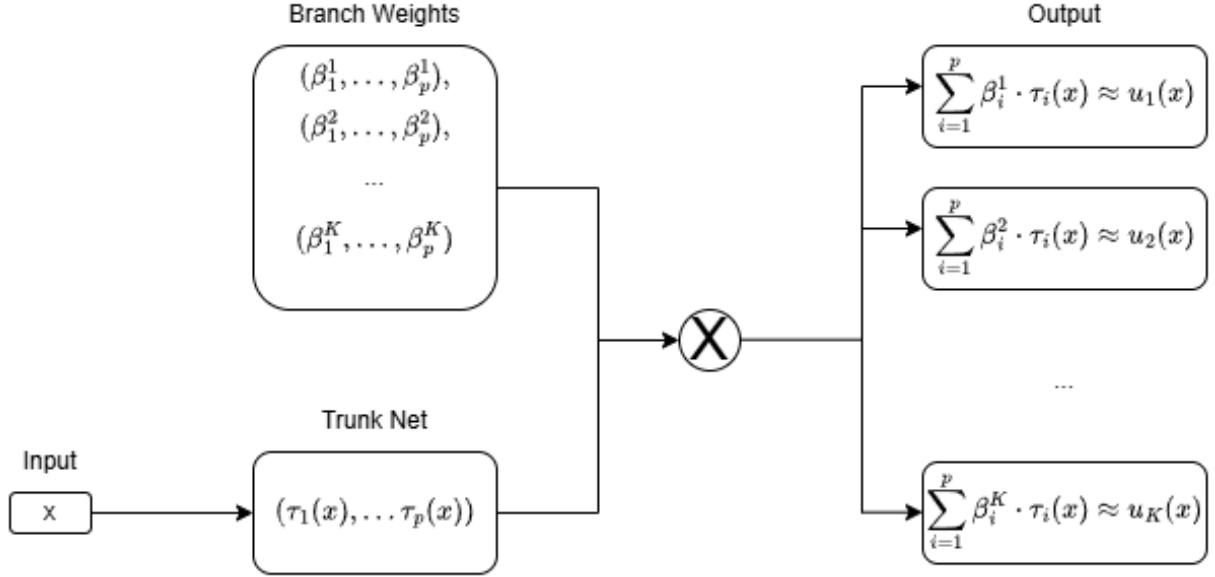


Figure 1: Illustration of the architecture of the Deflation-PINN.

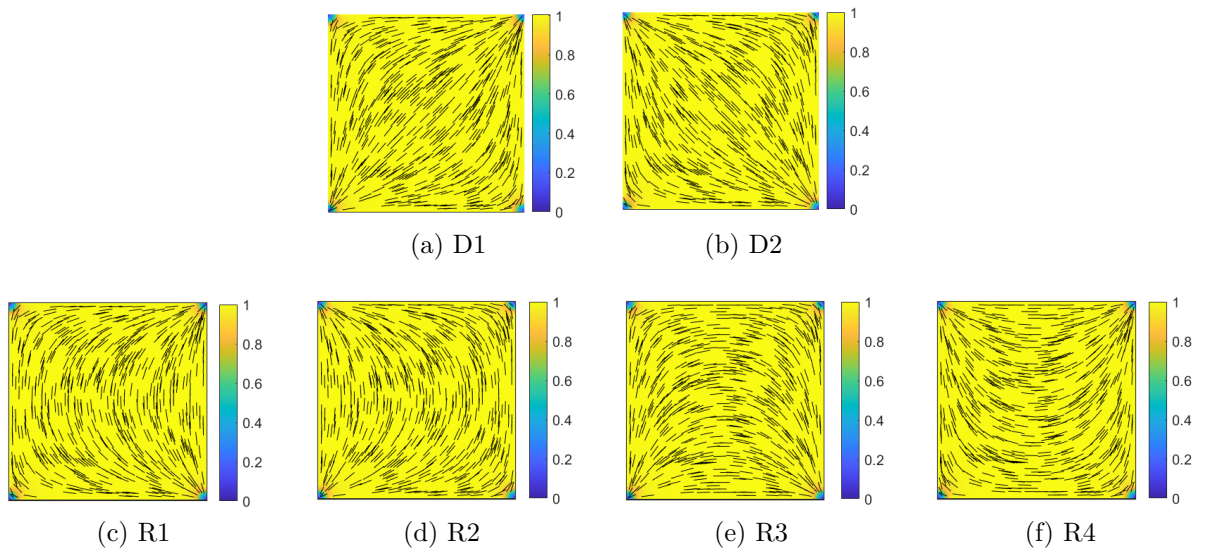


Figure 2: Diagonally stable molecular alignments: (a) D1, (b) D2 states and rotated stable molecular alignments: (c) R1, (d) R2, (e) R3, (f) R4 states.

Solutions	Relative L^2 Differences
D1	0.3752
D2	0.3681
R1	0.2767
R2	0.2786
R3	0.4008
R4	0.4089

Table 1: The table shows the relative $L^2([0, 1]^2, \mathbb{R}^2)$ distance of the respective solutions $(x, y) \mapsto (Q_{11}(x, y), Q_{12}(x, y))$ for the LDG problem from above computed with FEM and with the Deflation-PINN.

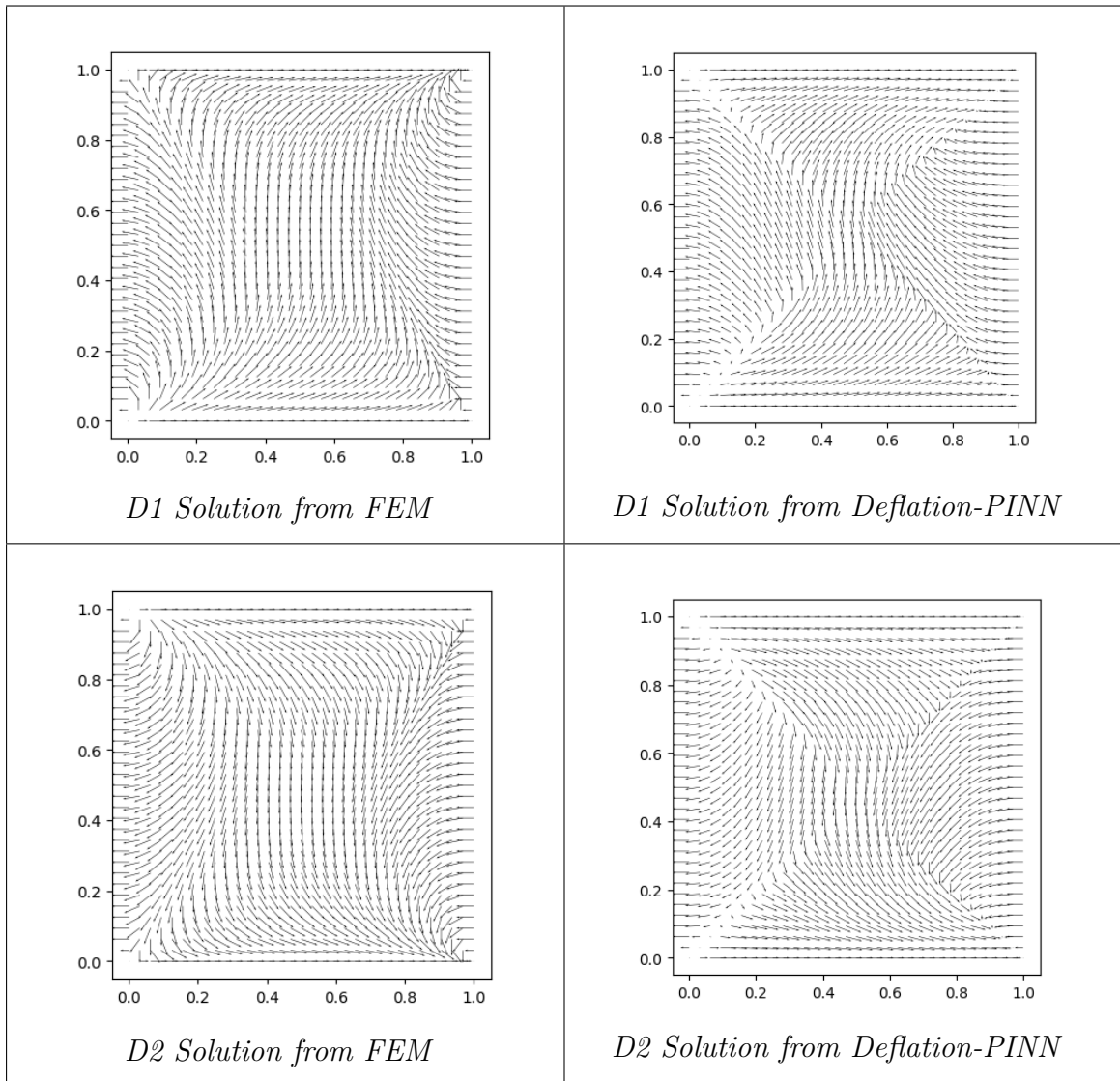


Table 2: Plots of the solutions D1 and D2 of the LdG problem with FEM and with Deflation PINNs.

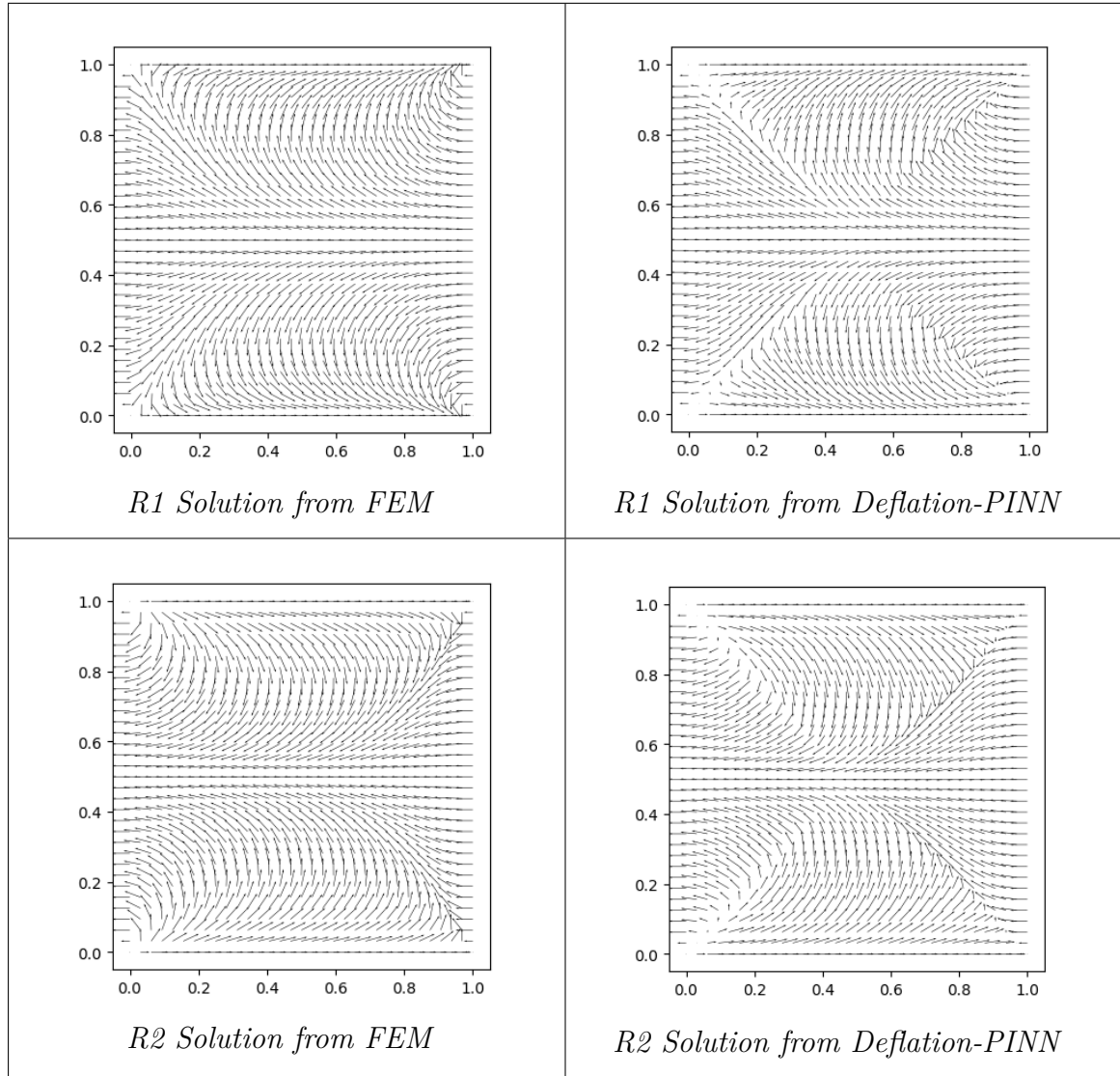


Table 3: Plots of the solutions R1 and R2 of the LdG problem with FEM and with Deflation PINNs.

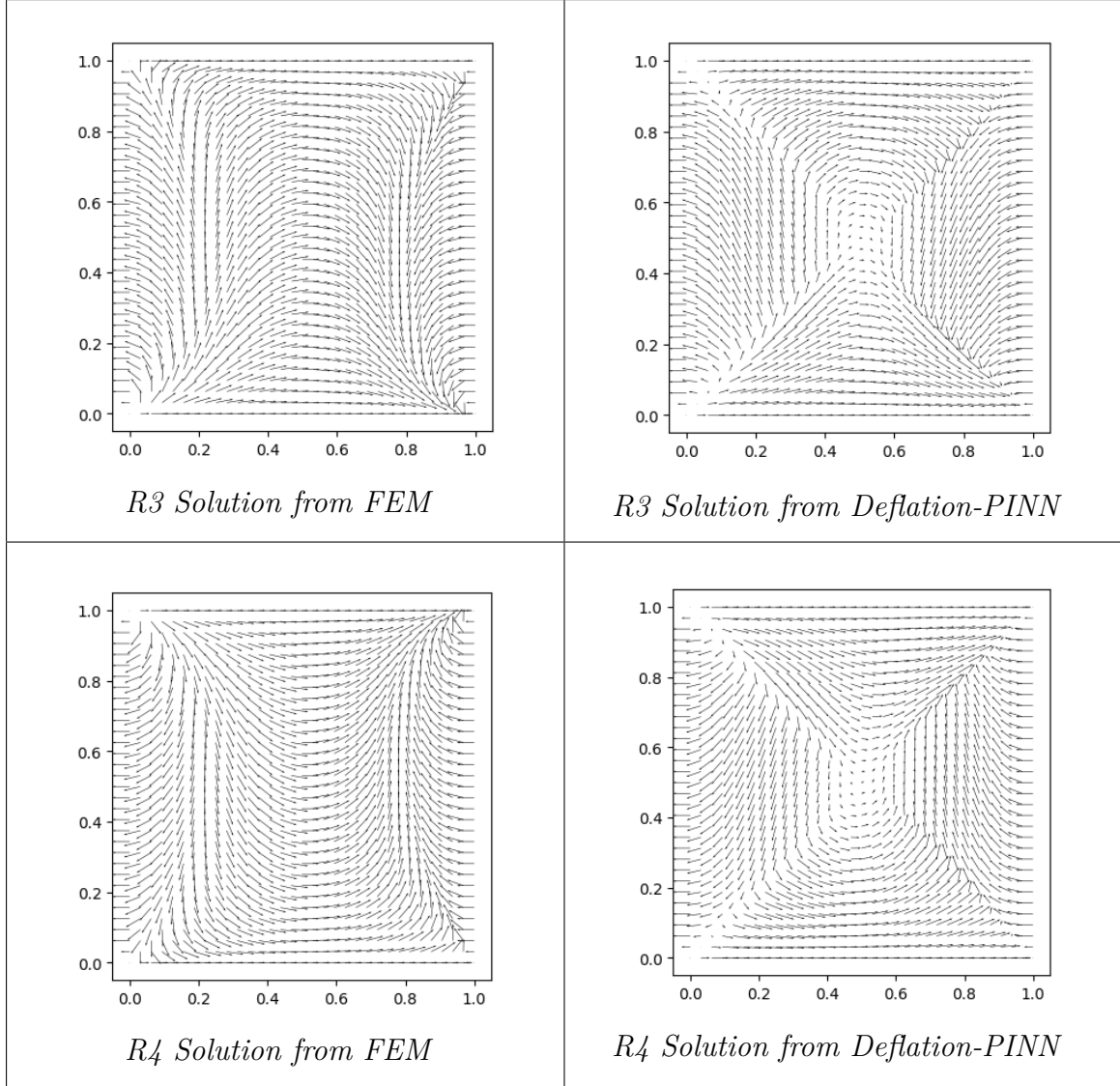


Table 4: Plots of the solutions R3 and R4 of the LdG problem with FEM and with Deflation PINNs.